

Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime

Gianluca Bresolin
gianluca.bresolin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Tommaso Prandin
tommaso.prandin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Abstract

This paper presents the implementation and analysis of a real-time demonstrator originally developed in *Ada* using the *Ravenscar Profile*, re-implemented in *Rust* using the *RTIC* framework. The demonstrator is designed to run on an *STM32F4* microcontroller and uses a *Fixed-Priority Scheduling* policy combined with *Stack Resource Policy (SRP)* for resource sharing. The analysis is performed using the *MAST* tool to verify timing properties and analyze system performance under various workloads. The results demonstrate the feasibility of adopting *Rust* and *RTIC* for real-time applications, while also highlighting differences and considerations compared to the original *Ada* implementation.

CCS Concepts

• **Software and its engineering** → **Real-time systems software**; **Embedded software**; *Software architectures*.

Keywords

Real-Time Kernels, Real-Time Systems, Embedded Software, Software Architectures

ACM Reference Format:

Gianluca Bresolin and Tommaso Prandin. 2025. Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime. In *Proceedings of Real-Time Kernels and Systems Exam (RTKS '25)*. ACM, New York, NY, USA, 1 page. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Real-time embedded systems demand precise control over timing, concurrency and resource management. Such systems are often used in critical applications where failure to meet timing constraints can lead to severe consequences. Hence, developing a real-time system requires careful consideration of the underlying runtime environment, necessitating a clear understanding of how the system will behave in order to provide sufficient guarantees on meeting timing requirements.

The choice of the runtime environment is a crucial aspect in the design of a real-time system, as it can significantly impact the system's ability to meet its desired behaviour through reliable mechanisms

it offers. One of the most popular languages that meets the needs of real-time systems is *Ada*, which has been widely used in the development of safety-critical systems due to its deterministic constructs that allow high analysability of the code. While *Ada* remains a robust choice, recent advances in the *Rust* programming language have introduced new opportunities for real-time systems development.

In this paper, a real-time system originally implemented using the *Ada Ravenscar Profile* is re-implemented in *Rust* using the *Real-Time Intrupt-driven Concurrency (RTIC)* framework, which is responsible for managing the scheduling of tasks through a *Fixed-Priority Scheduling* and the resource sharing in the system under *Stack Resource Policy (SRP)*.

For the analysis of the system, the *MAST* tool is used to verify the timing properties and to analyze the system under different workloads, which is run on a *STM32F4* microcontroller.

The goal of this work is to demonstrate the feasibility of using *Rust* and *RTIC* for real-time systems, while also providing some considerations and comparison with the original *Ada* implementation.

The rest of the paper is organized as follows. Section 2 presents a formal description of the system model. Section 3 introduces the *RTIC* framework and its key concepts. Sections 4 and 5 discuss the measurement of execution times and the detection of deadline misses, respectively. Section 6 provides an overview of the *MAST* tool and its analysis capabilities. In Section 7, we present the *Fixed-Priority Scheduling (FPS)* analysis of the system. Finally, Section 8 concludes the paper reporting the main findings.

2 System Model

3 RTIC

4 Execution Times

5 Deadline Miss Detection

6 MAST

7 FPS Analysis

8 Conclusions

9 Citations and Bibliographies

References

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RTKS '25, Padova, Italy

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>